The Steerability Challenge

Competition Guide

Submission deadline:	November 23, 2026 (AoE)
Workshop location:	NeurIPS 2026 (Paris)
Website:	`https://steerability.github.io/competition/`
Code repo:	`https://github.com/generative-computing/steerability`
Registration form:	`https://tally.so/r/ZjBg10`
Discord server:	`https://discord.gg/wcfXuEJK8M`
Document version:	`v0.6` (Nov 7, 2026)
Changes in v0.6:	Section 3.3 (model organisms) added; Sections 3.1, 4.1, 4.2, and 4.3 expanded with evaluation details (token ceilings and the request timeout, LoRA adapters on vLLM, the method card, size limits, the deadline rule, the carry-over of unchanged submissions between weekly evaluations); competition close moved to November 23, 2026 (AoE), and the winners are announced at the workshop; Starter Kit release 0.4.0.

Live document

Please note that this is a live document and will be updated with additional detail and clarification throughout the competition as needed.

1 Welcome!

Thank you for your interest in participating in The Steerability Challenge! This document outlines how to get started with the toolkit, how to build a submission (and submit it), details of the competition timeline, and some general tips for creating strong submissions. For any questions you may have that are not addressed by this document, please join our Discord server and ask in the `ask-the-organizers` channel, where we will be happy to help you out.

1.1 Motivation

Language models sometimes state things that contradict what they know to be true, i.e., they are dishonest. Given that language models are starting to be used in high stakes environments (e.g., medicine, law, etc.), the hazards associated with model dishonesty can be significant. Model steering can be an effective mechanism for improving honesty, however, how to best build these interventions is still an active area of research. There are various ways in which we can steer a model's behavior (e.g., prompting, activation steering, weight-based methods), but even the most effective methods still yield a considerable amount of observable dishonesty.[1] Furthermore, it appears that the mechanisms that influence a model's (dis)honesty are tightly coupled with those that dictate other (desirable) behaviors, thus steering for honesty may very well degrade a model's general capabilities.

1.2 Our Competition

Our competition focuses specifically on how to reduce model dishonesty while maintaining the model's general capabilities. To help provide structure to this challenge, and to facilitate standardized submissions and evaluations, the competition will be run using the `Steerability` toolkit.[2] One of the core abstractions in the toolkit is the notion of a *steering pipeline*, i.e., an ordered composition of steering methods (hereafter referred to as *controls*) that span modifications to the model's input (prompting), structure (weights), state (activations, attentions), and output (decoding). In addition to functionality for building steering pipelines, the toolkit also allows for running evals (via the `Inspect`[3] library) and probing model internals.

[1] Wang, Treutlein, Roger, Hubinger, and Marks, *Evaluating honesty and lie detection techniques on a diverse suite of dishonest models*, Alignment Science Blog, November 2025.

[2] `https://github.com/generative-computing/steerability`

[3] `https://inspect.aisi.org.uk`

Submissions will take the form of frozen steering pipelines (`.spipe` files) across our competition models (described in Section 3.1). Each steering pipeline contains the specification of the pipeline's structure (in `spipe.json`) as well as any trained artifacts the steering pipeline references (e.g., steering vectors). You can additionally define custom controls as part of your submission (e.g., a novel activation steering method). Section 4.1 contains additional information regarding the specific format of submissions.

2 Getting Started

2.1 Create Your Hugging Face Repository for Submissions

Submissions will be made through a private Hugging Face repository owned by your team. The competition organizers will be given read-only access to this repository in order to download and evaluate your submissions. To keep submissions private from other participants, each team must create its own Hugging Face organization. A paid Hugging Face plan is not required for this process.

1. *Create a Hugging Face account*

   If you do not already have a Hugging Face account, create one at `https://huggingface.co/join`.

2. *Create a new Hugging Face organization for your team*

   Go to `https://huggingface.co/organizations/new` and create a new organization (you can choose *Community* as the organization type). Please choose an organization username (and full name) that matches, or clearly identifies, your team name. This will be needed when you register your team. Hugging Face may prompt you to upgrade to a paid Team or Enterprise plan; please select "skip for now".

   We recommend using this organization only for this competition, since the competition evaluator will be granted read access to repositories within the organization.

3. *Add the competition evaluator to your organization*

   Open your organization's Hugging Face page and navigate to *Settings* → *Members*.

   Add our `steerability-challenge-evaluator` user as a Hugging Face user and assign this user the *Read* role. To do this, go to *Organization settings*, click *Members*, then *Add new member*.

   Please *do not* assign the evaluator `Contributor`, `Write`, or `Admin` permissions. Read access is sufficient for us to download and evaluate your submission. Additionally, please ensure that you add the evaluator *user account* `steerability-challenge-evaluator`, not the competition organization `steerability-challenge-evals`.

4. *Create a private repository for your submission*

   Go to `https://huggingface.co/new` and create a new model repository.

   When creating the model repository:

   - Set the *Owner* to the Hugging Face organization you created above.
   - Name the repository as "submission" (under *Model name*).
   - Set the repository visibility to *Private*.

   Your submission repository should therefore have a URL of the form

   `https://huggingface.co/<your-organization>/submission`.

   Please *do not create the submission repository under your personal Hugging Face account.* The repository must be owned by your team's organization so that the our evaluator can access it through its organization membership.

5. *Note your repository URL*

   Once your repository has been configured and your submission files have been uploaded, note the repository URL (`https://huggingface.co/<your-organization>/submission`). This repository URL will be required when registering your team.

   Please verify that:

   - the repository is *Private*
   - the repository is owned by your team's *Hugging Face organization*, rather than by a personal account
   - `steerability-challenge-evaluator` is a member of your organization with the *Read* role

2.2 Register Your Team

Once you have created your Hugging Face repository, the next step is registering your team via our Tally form (`https://tally.so/r/ZjBg10`). Teams can consist of anywhere from 1 to 5 people. We require that each participant on the team is associated with a verifiable identifier when registering (e.g., LinkedIn, Google Scholar, etc.). Please note that each participant is only allowed to be a member of one team (any violation of this rule will lead to disqualification of the entire team). Additionally note that there will be a short delay until we can validate your team's registration.

2.3 Install the Toolkit

Participants will build their submissions using the `Steerability` toolkit. The toolkit requires Python 3.12+ and is installed via the `uv` package manager (`https://docs.astral.sh/uv/getting-started/installation/`). We recommend installing the toolkit from source with all development extras so that you can read and experiment with the toolkit code. To install, please follow:

1. Clone the repository at `https://github.com/generative-computing/steerability` to your working directory (fork it first only if you plan to contribute changes back to the toolkit).
2. From the repository root, run `uv sync --extra all`. This creates a `.venv` (fetching a Python 3.12 interpreter if none is found), installs the toolkit in editable mode, and adds the `eval` extra (Inspect AI) together with the `dev` dependency group (pytest, pre-commit, and notebook functionality).

   Since the install is editable, you can read and step through the toolkit source. Controls that are part of your submission belong in your Starter Kit workspace under `controls/`, where they register themselves with `register_method`; this is the layout a submission uses (Section 4.1).
3. Activate the environment with `source .venv/bin/activate` (on Windows, `.venv\Scripts\activate`), or prefix each command with `uv run`.
4. Authenticate with the Hugging Face Hub via `hf auth login` (or export `HF_TOKEN`). Gated checkpoints such as `meta-llama/*` additionally require approved access on the model's Hub page.

To verify your installation please run the following:

```
>>> from steerability.algorithms.core.registry import REGISTRY
>>> {k: len(v) for k, v in REGISTRY.items()}
{'input_control': 6, 'output_control': 14, 'state_control': 8, 'structural_control': 6}
```

Importing the registry loads the toolkit and its core dependencies and discovers every registered control (`mergekit` is skipped unless the `merging` extra is installed, in which case `structural_control` counts 7). Then run a small subset of the test suite that builds a steering pipeline and generates on a tiny model:

```
pytest tests/controls/test_user_prefix.py
```

The first run downloads the test models listed in `tests/utils/ci_models.yaml` from the Hugging Face Hub. All tests should pass; tests for devices unavailable on your machine are skipped automatically.

Model access

Note that forks of public toolkits are public. As a result, any development work you do in the context of the competition should live in your private copy of the starter kit (see the next section for details).

2.4 Starter Kit

As part of our competition, we have prepared a Starter Kit to help you in building a submission. The Starter Kit contains some baseline steering pipelines, lightweight evaluation data (a released sample of the evaluation's own data, in its layout and row schema) and proxy judge functionality to sanity check your pipelines, and a submission checker script to allow you to validate that your submission is in the correct format before uploading; the checker also reports the track your submission is placed in and the inference engine each of your pipelines will be evaluated on (Section 4.3). The Starter Kit's `uv sync` installs the exact toolkit commit the evaluation runs (steerability 0.5.8 as of release 0.4.0 of the Starter Kit), so please build and check your submission in that

environment. Its `fetch-baselines` command downloads the organizer baselines (ten at the time of writing, listed in its README) from the public Hub repository `steerability-challenge-evals/baselines`. Once we have validated your team's registration, you will be invited to the private Starter Kit repository. In the meantime, we encourage participants to browse our example notebooks and documentation to get an understanding of the toolkit's functionality. The Starter Kit's README lists each release and its changes; please pull new releases into your workspace (see below), since they carry corrections that affect how your local numbers compare with the evaluation's.

Creating your private workspace. We intend for you to do most of your development in a private repository of your own, created from the Starter Kit. Please do not use GitHub's *Fork* button for this. Instead, once you have accepted the invitation, one member of your team should run the following:

```
git clone \
    https://github.com/steerability-challenge/steerability-challenge-starter-kit \
    my-team
cd my-team
git remote rename origin starter
gh repo create <owner>/<repo-name> --private \
    --source=. --remote=origin --push
```

The last command uses the GitHub CLI (`https://cli.github.com`; run `gh auth login` first if you have not yet authenticated) to create a new private repository under `<owner>`, which can be your GitHub username or an organization you control, and to push your local copy to it. If you prefer not to use the CLI, create an empty private repository in the GitHub web interface (without a README), then run `git remote add origin <url>` followed by `git push -u origin main`. Any private Git host works; GitHub is not required.

After this, `origin` is your team's private repository and `starter` is the read-only Starter Kit. Add your teammates as collaborators on your private repository and do your competition work there. This includes any custom controls for your submission, which belong in your workspace's `controls/` directory (Section 4.1). We will release additional baselines through the Starter Kit during the competition; you can pull them into your workspace at any time with `git pull starter main`.

If you are using agents to assist you, please point your agent to the `AGENTS.md` files in the root directories of both the toolkit and the Starter Kit, and instruct it not to push to, or open pull requests against, public repositories.

Compute requirements

Some of the steering interventions can be compute-heavy. Additionally, the competition will be focused on reasoning models in the range of 27–31B. We recommend that participants have access to a GPU with 80GB of VRAM.

2.5 Keeping Your Work Private

- Do your development in your private workspace (Section 2.4). Don't push competition work to a fork of the toolkit or to a GitHub fork of the Starter Kit.
- Keep your `submission` repository private (as explained in Section 2.1).
- Only add the evaluator account spelled exactly `steerability-challenge-evaluator`.
- Our Discord server and the toolkit's GitHub issues are public. Share minimal reproductions there rather than your method. For any questions that would reveal your approach, feel free to send a direct message (to `erik_jm.`) on Discord.
- If you use a coding agent, instruct it not to push to or open pull requests against public repositories, and not to create public repositories, gists, or Hub repositories.
- Winning teams will be asked to give organizers read access to their repository for the code review (Rule 4). After results are announced, we encourage public release.

3 The Task

The goal of the competition is to reduce model dishonesty while minimizing degradation of (off-target) general capabilities. To avoid overfitting, we do not reveal the specific measures of dishonesty or general capabilities, however, at a high-level you can assume that dishonesty covers lying and sycophantic behaviors (the fourteen dishonesty dimensions described in the Starter Kit's *Dishonesty Dimensions* document, plus the model organisms of Section 3.3) whereas general capabilities measure knowledge, reasoning, problem solving (including code), fidelity (e.g., copying or transforming a text faithfully, recalling a fact from a long context), and multilingual ability. A set of *guards* additionally checks that honesty was not gained by degrading something else (e.g., instruction following or helpfulness) and that it was not overshot into refusal, evasion, or disclaimers. In general, your submissions will be scored on (i) the improvement it produces over the unsteered model on a collection of evaluations that elicit dishonest behavior, and (ii) the regressions it causes on a suite of general-capability benchmarks and on the guards. These will be combined into a single composite score which will be posted on the leaderboard. The Starter Kit's README states the composite and the per-candidate scoring in full, and its `data/` folder contains a released sample of the evaluation's own data (the dishonesty dims, the guards that have their own data, and the locally built capability dims), in the evaluation's layout and row schema.

3.1 Competition Models

Since we are evaluating the methodolody of your steering approach (not just a steered model), submissions will consist of steering pipelines, one per competition model below. We require that the pipelines only differ in terms of the trained artifacts that are referenced and not in the structure of the pipeline itself. The submission checker script provided in the Starter Kit validates this constraint. The intent is that you develop a *method*, not three unrelated artifacts that are overfit to each model.

We have selected the competition models based on their recency and overall performance. The competition models are:

1. `google/gemma-4-31B-it`
2. `ibm-granite/granite-4.2-30b`
3. `Qwen/Qwen3.8-27B`

Default inference parameters for the models (sampling, the Hub revision of each model, and the per-suite ceilings on new tokens, thinking included) are specified in the Starter Kit (the constants `REGIME`, `BASE_MODEL_REVISIONS`, and `MAX_NEW_TOKENS` of its `starterkit` package). Note that all of the above models are thinking models; thinking will be enabled but the actual thinking trace will not be used in the evaluations. A continuation that reaches its token ceiling, or whose reasoning block never closes, scores 0, which matters for methods that make the model think longer. At the weekly evaluations the ceiling is 8192 tokens (thinking included) on the dishonesty dims and most guards and 16384 on the capability benchmarks and a few guards. Additionally note that sampling parameters are not part of a `.spipe`; if a submission requires extra sampling parameters they should be expressed through a control (e.g. `best_of_n`).

Each submission runs without network access on two 80 GB GPUs. Each generation request must finish within the evaluation's timeout of three hours; a request past it fails its sample, the suite is retried a fixed number of times, and a sample that still fails leaves the submission unscored on that model for the interval. The evaluation runs a bundle on vLLM where the toolkit's backend check (`pipeline.check()`) passes there and on Hugging Face (`bfloat16`) otherwise, and pairs every score with the unsteered reference run on the same engine (Section 4.3); the Starter Kit's checker reports the engine each of your bundles will run on.

3.2 Competition Tracks

We have defined three broad tracks for the competition.

1. **Black-box**: As the name suggests, the black-box track consists of pipelines that neither require access to nor modify the model's internals (e.g., weights, activations, or logits).
2. **White-box**: Pipelines in the white-box track are allowed to both access and modify the model's states and logits.
3. **Open**: Pipelines in the open track are able to additionally modify the weights of the model.

Tracks nest in the sense that everything allowed in the black-box track is allowed in the white-box track, and everything allowed in the white-box track is allowed in the open track. If a submission spans two tracks, then the submission competes in the higher one. Given that only one submission per team is scored (at the evaluation intervals), a team's submission will only ever compete in a single track.

The identification of a track is categorized automatically based on what parts of the steering pipeline accesses and modifies. Special tracks in addition to the ones above (e.g., based on novelty, efficiency, etc.) may be created depending on the pool of submissions.

3.3 Model Organisms

In addition to the dishonesty dimensions, every pipeline is also applied to *model organisms*, i.e., copies of the competition models that have been fine-tuned into a dishonest behavior. The current organisms have been fine-tuned to lie in answer to factual questions (one organism per competition model); further behaviors may be added for the final evaluation. The organisms' weights are held out and are not released; the competition website provides an endpoint for querying the lying organisms within a limited budget Each organism is evaluated the same way as its base model: your pipeline is installed on top of the organism's weights (a LoRA adapter composes with them as with the base model), its responses are judged, and the result is scored against the organism's own unsteered run, so what is measured is how much of the trained-in dishonesty your method removes. Per competition model, the dishonesty term of the composite score is a weighted combination of the dishonesty-dimension term and the organism term, with the organism term weighted at one third; the organisms' guard regressions enter the regression terms in the usual way. Organism-based evaluations have been part of the leaderboard scores since the fifth weekly evaluation.

4 Preparing Your Submission

4.1 What is a Submission?

A submission is a directory containing:

- Steering pipelines (`.spipe` files): Three `.spipe` files (one per competition model), each frozen (saved with `freeze=True`), self-contained (saved with `artifacts="fat"`), named `<model-slug>.spipe`, and at most 5 GB.
- Method card (`method_card.md`): A short Markdown description of the method with the sections *Summary*, *Controls*, and *Data* (the Starter Kit's README gives the template). The organizers read it during the review; nothing in it is parsed, except an optional front matter (`name`, `version`, `summary` between two `--` lines), which is used as the display label of your submission. A missing or empty card is currently a warning, not a failure.
- Custom controls (optional, `controls/`): Any custom controls as Python source code that the `.spipes` make use of. Note that if these controls require any steering artifacts (e.g., steering vectors) then these must be provided in the `.spipe` files. Additionally, `controls/__init__.py` must register each control with `register_method`; the evaluator imports this package before loading your `.spipe` files. A submission with custom code is run and scored as usual and appears on the leaderboard once the organizers have reviewed the code and assigned its track.

Nothing else in the directory may be code: a symlink, an extra directory, or a file outside `controls/` that can run or be imported (`.py`, `.sh`, `.so`, and the like) fails intake, while any other extra file (a `README.md`, a `LICENSE`, notes) is ignored. The Starter Kit contains a script that verifies that the submission is properly formed before you submit it, including this layout rule.

LoRA adapters on vLLM

A bundle whose controls all pass the vLLM backend check (prompt-only pipelines and LoRA adapters, for instance) is run on vLLM, which serves a LoRA adapter only as a plain PEFT adapter: the rank `r` (including any `rank_pattern` rank) must be at most 64, the adapter may not use per-module `rank_pattern` or `alpha_pattern` values (vLLM reads neither), DoRA, an adapter bias, or `modules_to_save`, and its `adapter_config.json` must be readable. An adapter that violates any of these fails intake rather than being served incorrectly. Release 0.4.0 of the Starter Kit's `check-submission` applies the same checks.

4.2 Uploading Your Submission

Submissions will be made using your Hugging Face repo specified in Section 2.1. You may upload files directly using the Hugging Face web interface or using Git. To upload using Git, first install the Hugging Face command-line tools:

```
pip install -U hf
```

Authenticate with your Hugging Face account and configure your credentials for Git:

```
hf auth login --add-to-git-credential
```

Then clone your submission repository:

```
git clone https://huggingface.co/<your-organization>/submission
cd submission
```

If your submission contains large files, such as model weights, you should also install Git-Xet as described in the Hugging Face documentation (`https://huggingface.co/docs/hub/repositories-getting-started`).

Lastly, copy the required submission files into the repository and push them:

```
git add .
git commit -m "Upload submission"
git push
```

You may update your repository as many times as you wish. At each evaluation interval we download the repository at its current (latest) commit. Please ensure that your repository only ever contains one submission in order to avoid the wrong submission being evaluated, and that it stays within the size limits (5 GB per bundle and about 15.5 GB for the whole repository; a larger repository is not downloaded). For the final evaluation, the repository's latest commit must be made by the deadline (November 23, 2026, AoE): a repository whose latest commit is after the deadline is recorded as late and is not downloaded, so please do not push to it after the deadline.

4.3 Evaluation Intervals

Submissions are evaluated weekly and at competition close. The depth of these evaluations varies, as discussed below. Every evaluation samples several candidates per prompt, and every score on the leaderboard carries a bootstrap confidence interval.

Evaluations run on two inference engines. Each pipeline is run on vLLM where the pipeline allows it, i.e., where the toolkit's backend check (`pipeline.check()`) reports that every control is supported there, and on Hugging Face (`bfloat16`) otherwise; for example, prompt-only pipelines and LoRA adapters run on vLLM, whereas a pipeline containing a control that needs in-process access to the model, such as activation steering, runs on Hugging Face. Since the two engines produce slightly different outputs for the same weights, every submission is scored against the unsteered reference arm run on the same engine, so the routing does not favor either engine. A submission that neither engine can run fails intake. The Starter Kit's submission checker reports which engine each of your pipelines would run on, and its generation helpers can run your pipelines on either engine with the evaluation's settings.

Weekly. The weekly evaluations run the dishonesty dimensions, the guards, and the organisms on a fixed subset of the private evaluation data, and the capability benchmarks on smaller seeded subsets than the final; the leaderboard is updated using these scores. A submission that is unchanged from one weekly evaluation to the next uses its earlier results. A submission with any changed file is regenerated in full.

Final. The final evaluation is the most extensive one and is authoritative in the sense that it overrides previous evaluations. It runs every private row of every dimension and guard, including rows in held-out prompt styles that no weekly evaluation runs, the full capability subsets, and the organisms, with the same generation settings and token ceilings as the weekly evaluations. Every submission is regenerated from scratch at the final.

5 Competition Timeline

Important dates:

1. *September 24, 2026*: Weekly evaluation of submissions begins with the results uploaded to the leaderboard.
2. *October 19, 2026*: Competition midpoint. The organism-based evals (Section 3.3) are incorporated into the computation of the leaderboard scores from the week-5 evaluation (published November 5, 2026), from which the full set of dishonesty dimensions and guards is also in place. The organisms themselves are held out and are not released.
3. *November 23, 2026*: Competition close. Participants will have until November 23 AoE to make a final submission upload.
4. *December 13, 2026*: Competition workshop at NeurIPS 2026 in Paris, where the winners will be announced.

The first two weeks of the competition are reserved for participants to get started with the toolkit. Throughout the competition, we will continue to develop and release our own baseline steering pipelines and share them with participants. These will serve as local optima and are intended to be improved upon by the participants. Note that given the short timeline between the competition close and the workshop date we do not require in-person attendance.

6 Building Steering Pipelines

6.1 General Guidance

One of the unique functionalities in the toolkit is the ability to compose multiple controls into a steering pipeline, i.e., an ordered set of controls either from the same control category (input, structure, state, and output) or across categories (e.g., prompting combined with activation steering). Anecdotally, we have found that combining controls can be quite effective due to the ability to exploit complementary strengths across different methods. We encourage participants to explore building pipelines that contain strictly more than one type of control.

Additionally, we encourage participants to make use of our `Probe` functionality to access the model's hidden states. Hidden states can be very informative for certain behaviors, especially those that are more ingrained in a model's behavior (e.g., dishonesty).

6.2 Other Tips

We've compiled some additional tips for steering. Note that we aim to expand on this list throughout the competition.

- For activation steering, the layer at which to steer matters a lot. Thus, we recommend either using the same layer as the baselines or conducting your own layer sweep to identify the best layer for steering input.
- The quality of the contrastive samples has a large effect on the resulting steering direction, so always ensure that you filter the data before building your method. In particular, we recommend checking that the model actually exhibits the labeled behavior in each sample (e.g., that a response labeled as dishonest is in fact dishonest), and discarding samples that are ambiguous or that differ from their contrastive counterpart in ways unrelated to honesty. Since the direction is typically extracted from a small number of samples, even a few mislabeled or noisy samples can noticeably shift it.
- The `gemma-4` model family tends to have very large activations for some tokens (especially in the prompt). To avoid degrading the model's capabilities during activation steering, we recommend either steering only the response tokens or implementing a gating mechanism that prevents steering tokens with very large activations.
- If you are building an activation steering method and find that the model coherence is sensitive to steering, trying steering with a lower magnitude across multiple layers (more than one).
- The type of contrastive samples matters more than their quantity. We've observed that it is possible to extract an activation steering vector from only 10 contrastive samples. What determines whether this direction is effective at restoring honesty and generalizes across many out-of-distribution settings is the scenario the samples are drawn from, i.e., the situation that elicits the dishonest behavior (for example, whether the model is explicitly instructed to lie, is under pressure to conceal information, or is agreeing with a user's false belief), rather than how many samples are used.

- Some dimensions are parsed rather than judged. For example, a multiple-choice letter on the last line, a `Rating: N/10` line, or a `Confidence: N%` line (the *Dishonesty Dimensions* document gives the exact rules, and `starterkit.parse` restates the parsers). A method that changes the shape of the answer segment (e.g., a decoding control that appends text after the final line, or Markdown emphasis around the letter) can turn an answered candidate into an unanswered one so please check the parsed output of your pipeline on those dimensions.
- The evaluation is two-sided in the sense that several guards penalize overshooting honesty into refusal, evasion, or disclaimers, relative to the unsteered model. The sample guard files in the Starter Kit's `data/guards/` show what each of them checks.

7 Communication

If you have any questions about any of the above or encounter any issues, please ask in the `ask-the-organizers` channel on our Discord server (or open an issue on our GitHub page for toolkit bugs). Questions asked there are answered where every team can see them; for a question that would reveal your approach, send a direct message as described in Section 2.5. We look forward to seeing what you build!